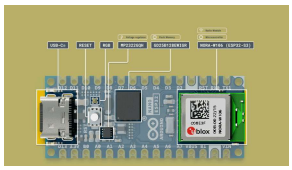
Arduino Nano ESP32 User Manual

A technical summary of the Nano ESP32 development board, including installation, pin reference, communication ports and microcontroller specifics.

Author · Karl Söderby & Jacob Hylén Last revision · 02/19/2025

The **Arduino Nano ESP32** is the first Arduino to feature an ESP32 SoC as its main microcontroller, based on the ESP32-S3. This SoC is found inside the **u-blox® NORA-W106** module and provides both Bluetooth® & Wi-Fi® connectivity, as well as embedding an antenna.

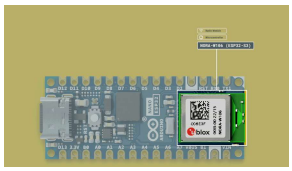


Nano ESP32 overview

In this document, you will find information regarding features of the board, and links to resources.

> Note that this board is compatible with many ESP32 examples out of the box, but that the pinout may vary. You can find the complete API at ESP32-S3 API reference.

NORA-W106 (ESP32-S3)



NORA-W106 module.

The Nano ESP32 features the **ESP32-S3** system on a chip (SoC) from Espressif, which is embedded in the **NORA-W106**

bit LX7, and has support for the 2.4 GHz Wi-Fi® band as well as Bluetooth® 5. The operating voltage of this SoC is 3.3 V.

The NORA-W106 also embeds an antenna for Bluetooth® and Wi-Fi® connectivity.

Memory

The Nano ESP32 has

- 384 kB ROM
- 512 kB SRAM
- 16 MB of Flash (external, provided via GD25B128EWIGR)
- 8 MB of PSRAM

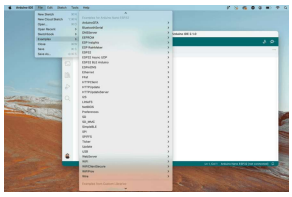
Datasheet

The full datasheet is available as a downloadable PDF from the link below:

Download the Nano ESP32 datasheet

Arduino ESP32 Board Package

This board is based on the **Arduino ESP32 Board Package**, that is derived from the original ESP32 Board Package. It provides a rich set of examples to access the various features on your board, which is accessed directly through the IDE.



ESP32 examples in the IDE.

To install the Board Package, go the **board manager** and search for **Nano ESP32**. For more detailed instructions to install the Board Package, please refer to the Getting Started with Nano ESP32 article.

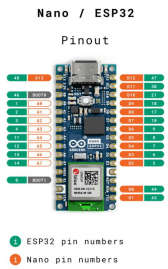
ESP32 Pin Map

The Nano ESP32's default pins are designed to match the **Nano form factor**. This pin mapping is done in the official Arduino ESP32 Board Package (see just above). See below the pin map to understand how the physical pins correlate to the ESP32:

Nano	ESP32
D0	GPIO44
D1	GPIO43
D2	GPIO5
D3	GPIO6
D4	GPIO7
D5	GPIO8
D6	GPIO9
D7	GPIO10
D8	GPIO17
D9	GPIO18
D10	GPIO21

Nano	ESP32
D12	GPIO47
D13	GPIO48
A0	GPIO1
A1	GPIO2
A2	GPIO3
A3	GPIO4
A4	GPIO11
A5	GPIO12
A6	GPIO13
A7	GPIO14
BOOT0	GPIO46
BOOT1	GPIO0

See the pinout below for a better visual translation:



Nano / ESP32 pinout

Arduino Bootloader Mode

The Nano ESP32 has a feature that we call Arduino Bootloader-mode, what this means is that you are able to put the board in a sort of recovery mode by double pressing the reset button while the board is powered on.

This mode is useful if you've

some unwanted behaviour. Maybe the sketch causes it to become undetectable by your computer, or maybe its an HID sketch that took over your keyboard and mouse and you need to regain control of your computer. It lets you turn the board on without actually running any sketch.

To enter bootloader-mode, press the reset button, and then press it again once you see the RGB LED flashing. You'll know that you've successfully entered bootloader-mode if you see the green LED pulsing slowly.

> Some boards from the first limited production batch were assembled with a different RGB LED which has the green and blue pins inverted. Read our full Help Center article here

ROM Boot Mode

In addition to the normal bootloader-mode, the Arduino Nano ESP32 lets you enter ROM boot mode. This is rarely needed, but there are some cases where it might be useful, for example you may want to follow this process to:

- Update the Arduino bootloader already on the board. This can resolve issues with Nano ESP32 being misidentified as other ESP32 boards.
- Restore the ability to upload regular Arduino sketches to

flashed with a third party firmware.

If you need to reflash the bootloader, you can follow the steps of this **Help Center article**

Default Sketch

The default sketch loaded on the Nano ESP32 board is found in the code snippet below:

```
void setup() {
  // put your setup c
  pinMode(LED_RED, OU
  pinMode(LED_GREEN,
  pinMode(LED_BLUE, O
  pinMode(LED_BUILTIN
}

void loop() {
  digitalWrite(LED_BU
  digitalWrite(LED_RE
  digitalWrite(LED_GR
  digitalWrite(LED_BL

  delay(1000);

  digitalWrite(LED_BU
  digitalWrite(LED_RE
  digitalWrite(LED_GR
  digitalWrite(LED_BL

  delay(1000);

  digitalWrite(LED_BU
  digitalWrite(LED_RE
  digitalWrite(LED_GR
  digitalWrite(LED_BL

```

MicroPython

The Nano ESP32 has support for

can easily be installed on your board.

To get started with MicroPython, please visit MicroPython 101, a course dedicated towards learning MicroPython on the Nano ESP32.

In this course, you will fundamental knowledge to get started, as well as a large selection of examples for popular third-party components.

Reset Your Board

If you have installed MicroPython but wish to go back to classic Arduino / C++ programming, it is easy to do so. Simply **double tap** the **RESET** button on the board (there's only one button). The board will enter boot mode (you should see a pulsing green light), and will be visible in the Arduino IDE.

Arduino Cloud

Nano ESP32 is supported in the Arduino Cloud platform. You can connect to the Cloud either through "classic" Arduino, using the C++ library, or via MicroPython:

- Getting Started with Arduino Cloud (classic)
- MicroPython with Arduino Cloud

API

The Nano ESP32 can be programmed using the same API as for other Arduino boards (see language reference).

However, the ESP32 platform provides additional libraries and built-in functionalities that may not available in the standard Arduino API.

For more information, see the ESP32-S3 API

Peripherals API

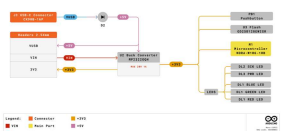
To learn more about the ESP32-S3's peripherals (e.g. ADC, I2C, SPI, I2S, RTC), refer to the Peripherals API section.

Sleep Modes

The Nano ESP32 can be programmed to draw a minimal amount of power, making it suitable for power constrained designs, such as solar/battery powered projects.

The Sleep Modes section in the ESP32 docs explains how to configure your board to draw minimal power, introducing the **light sleep** and **deep sleep**

Power Considerations



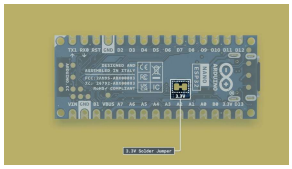
Nano ESP32 Power Tree.

To power the Nano ESP32 you may either use a USB-C® cable, or the VIN pin. When using the VIN pin, use voltages within the range of 5-18 V as the **MP2322GQH** converter on the board may otherwise be damaged.

Input Voltage (VIN)

> If you're using the USB-C® connector you must power it with 5 V.
>
> The recommended input voltage on the VIN pin is 6-21 V.

If you flip the board to view its underside, you'll find a solder jumper labelled "**3.3V**". If you cut the small trace between the two pads, you disconnect the step-down converter from the board, and your board will no longer turn on when plugged in to the USB port, or when its powered through the VIN pin. Instead you must provide **exactly** 3.3 V directly to the 3.3 V pin of your board. This can, depending on your power source, be a more energy efficient method of powering your board than powering through the VIN pin or the USB port.



3.3 V Solder Jumper

The internal operating voltage of the ESP32-S3 SoC is 3.3 V, and you should not apply voltages higher than that to the GPIO pins.

5V Pin / VBUS

The Nano ESP32 is the first board to not feature a **5V** pin. It has instead been replaced with VBUS, which is a more accurate description of the pin's capabilities.

`VBUS` provides 5 V whenever powered via USB. If powered via the VIN pin, it is disabled. This means that while powering the board through the VIN pin, you can't get 5 V from the board, and you need to use a logic level shifter or an external 5 V power supply.

This measure is taken to prevent the board's microcontroller from accidentally receiving 5 V, which will damage it.

Pins

The Nano ESP32 has two headers: the **analog** and **digital**. Listed here are the **default** pins that comply with previous Nano form factor designs.

The following pins are available on the board:

Pin	Type	Function
D13/SCK	Digital	**SPI** Serial Clock / LED

Pin	Type	Function
D12/CIPO	Digital	**SPI** Controller In Peripheral Out
D11/COPI	Digital	**SPI** Controller Out Peripheral In
D10	Digital	GPIO
D9	Digital	GPIO
D8	Digital	GPIO
D7	Digital	GPIO
D6	Digital	GPIO
D5	Digital	GPIO
D4	Digital	GPIO
D3	Digital	GPIO
D2	Digital	GPIO
D1/RX	Digital	GPIO 1 / **UART** Receiver (RX)
D0/TX	Digital	GPIO 0 / **UART** Transmitter (TX)
A0	Analog	Analog input 0
A1	Analog	Analog input 1
A2	Analog	Analog input 2
A3	Analog	Analog input 3
A4	Analog	Analog input 4 / **I2C** Serial Datal (SDA)
A5	Analog	Analog input 5 / **I2C** Serial Clock (SCL)
A6	Analog	Analog input 6
A7	Analog	Analog input 7

Note that all pins can be used as GPIO, due to the ESP32's flexibility.

Digital

The Nano ESP32 has 14 digital pins (D0-D13), that can be read by using `digitalRead()` or written to using `digitalWrite()`.

Pin	Type	Function
D13/SCK	Digital	**SPI** Serial Clock / LED Built in
D12/CIPO	Digital	**SPI** Controller In Peripheral Out
D11/COPI	Digital	**SPI** Controller Out Peripheral In
D10	Digital	GPIO
D9	Digital	GPIO & RX1
D8	Digital	GPIO & TX1
D7	Digital	GPIO
D6	Digital	GPIO
D5	Digital	GPIO
D4	Digital	GPIO
D3	Digital	GPIO
D2	Digital	GPIO
D0/RX	Digital	GPIO 0 / **UART** Receiver (RX0)
D1/TX	Digital	GPIO 1 / **UART** Transmitter (TX0)

Note that all analog pins can be used as digital pins as well, but not vice versa.

Analog

There are 8 analog input pins on the Nano ESP32, with 2 reserved

ADCs, `ADC1` and `ADC2`, where each ADC uses 4 channels each.

The default resolution for the ADC pins are 12-bit (returns a value between 0-4095).

Pin	Type	Function	ADC channel
A0	Analog	Analog input 0	`ADC1_CH0`
A1	Analog	Analog input 1	`ADC1_CH1`
A2	Analog	Analog input 2	`ADC1_CH2`
A3	Analog	Analog input 3	`ADC1_CH3`
A4	Analog	Analog input 4 / **I2C** Serial Datal (SDA)	`ADC2_CH1`
A5	Analog	Analog input 5 / **I2C** Serial Clock (SCL)	`ADC2_CH2`
A6	Analog	Analog input 6	`ADC2_CH3`
A7	Analog	Analog input 7	`ADC2_CH4`

Please note that `ADC2` is also used for Wi-Fi® communication and can fail if used simultaneously.

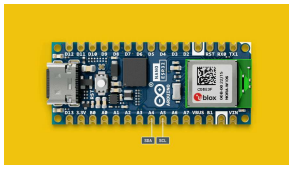
For more details, see **Analog to Digital Converter (link to Espressif docs)**.

Pulse width modulation (PWM) is supported on **all digital pins (D0-D13)** as well **as all analog pins (A0-A7)**, where the output is controlled via the `analogWrite()` method.

```
analogWrite(pin,value);
```

Due to timer restrictions, only 5 PWM signals can be generated simultaneously.

I2C



I2C Pins

The default pins used for the **main I2C bus** on the Nano ESP32 are the following:

Pin	Function	Description
A4	SDA	**I2C** Serial Data
A5	SCL	**I2C** Serial Clock

To connect I2C devices you will need to include the Wire library at the top of your sketch.

```
#include <Wire.h>
```

initialize the I2C port you want to use.

```
Wire.begin() //SDA & SD
```

And to write something to a device connected via I2C, we can use the following commands:

```
Wire.beginTransmission(
Wire.write(byte(0x00));
Wire.write(val); //send
Wire.endTransmission();
```

Second I2C Bus

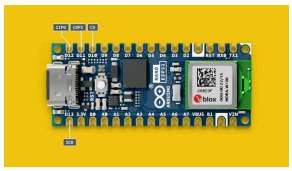
The Nano ESP32 has a second I2C bus, accessed via `Wire1`. To use it, you will need to set two free pins for SDA & SCL.

For example:

```
//initializes second I2
Wire1.begin(D4, D5); //
```

> `Wire` and `Wire1` can be used simultaneously, a great feature when working with devices that may share the same addresses.

SPI



SPI Pins

The Nano ESP32's SPI pins are listed below:

Pin	Function	Description
D10*	CS	Chip Select
D11	COPI	Controller Out, Peripheral In
D12	CIPO	Controller In, Peripheral Out
D13	SCK	Serial Clock

*Any GPIO can be used for chip select.

The following example shows how to use SPI:

```
#include <SPI.h>

const int CS = 10;


void setup() {
  pinMode(CS, OUTPUT);

  SPI.begin();

  digitalWrite(CS, LOW

  SPI.transfer(0x00);

  digitalWrite(CS, HIG
}

void loop() {
}
```

Second SPI Port (HSPI)

The Nano ESP32 has a second SPI port (HSPI). To use it, we need to create an object using `SPIClass`, and initialize communication on a specific set of pins.

> The HSPI port's default pins are: `GPIO14` (SCK), `GPIO12` (CIPO), `GPIO13` (COPI), `GPIO15` (CS). As some of these pins are not accessible on the Nano ESP32, you will need to configure them manually. See the definitions at the top of the code example below.

```
//define SPI2 pins man
//you can also choose
#define SPI2_SCK D2
#define SPI2_CIPO D3
#define SPI2_COPI D4
#define SPI2_CS D5

//create SPI2 object
SPIClass SPI2(HSPI);

void setup() {
//initialize SPI commu
  SPI2.begin(SPI2_SCK,
}
```

USB Serial & UART

The Nano ESP32 board features 3 hardware serial ports, as well as a port exposed via USB.

`Serial` refers to the USB port.

accessible via the board's RX/TX pins (D0, D1).

`Serial1` is the second UART port, which can be assigned to any free GPIOs.

`Serial2` is the third UART port, which can also be assigned to any free GPIOs.

Serial (Native USB)

Sending serial data to your computer is done using the standard `Serial` object.

```
1 Serial.begin(9600);
2 Serial.print("hello wor
```

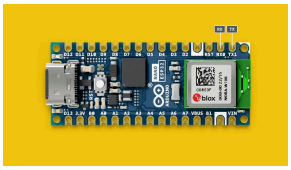
To send and receive data through UART, we will first need to set the baud rate inside `void setup()`.

Serial0 (UART)

Please note: `Serial0` is shared with the bootloader/kernel, which prints a few messages at boot/reset, and in the event of a crash, the crash dumps is printed via FreeRTOS on this serial port. For these reasons, you may want to use the `Serial1` or `Serial2` ports to avoid any interference (**read more**).

The default pins for UART communication on the Nano ESP32 are the following:

Pin	Function	Description
D0	RX	Receive Serial Data
D1	TX	Transmit Serial Data



Nano ESP32 UART Pins

To send and receive data through UART, we will first need to set the baud rate inside `void setup()`. Note that when using the UART (RX/TX pins), we use the `Serial0` object.

```
Serial0.begin(9600);
```

To read incoming data, we can use a while loop() to read each individual character and add it to a string.

```
while(Serial0.available
    delay(2);
    char c = Serial0.re
    incoming += c;
  }
```

And to write something, we can use the following command:

Serial1 & Serial2 (UART)

The Nano ESP32 features 2 additional hardware serial ports that have no pre-defined pins, and can be connected to any free GPIO. Therefore, to use them, their TX and RX pins need to be manually assigned.

To use `Serial1` and `Serial2`, you need to initialize them in your program's `setup()` function:

```
//initialization
Serial1.begin(9600, SER
Serial2.begin(9600, SER

//usage
Serial1.write("Hello wo
Serial2.write("Hello wo
```

Replace `RXPIN` and `TXPIN` with the GPIOs you want to assign (e.g. `D4`, `D5`).

You can then use commands such as `Serial1.write()` and `Serial1.read()`.

The `SERIAL_8N1` parameter is the configuration for serial communication.

`8` = data word length (8-bit). Can be changed to 5,6,7-bits.

`N` = parity, in this case "none". Can be changed to "even" (E) or "odd" (O).

`1` = stop bit, other option available is 2.

The Inter-IC Sound (I2S or IIS) protocol is used for connecting digital audio devices with a variety of configurations (Philips mode, PDM, ADC/DAC).

The default pin configuration for I2S is:

Pin	Definition
D7	`PIN_I2S_SCK`
D8	`PIN_I2S_FS`
D9	`PIN_I2S_SD`
D9	`PIN_I2S_SD_OUT`
D10	`PIN_I2S_SD_IN`

The default pins can be changed by using the `setAllPins()` method:

```
1  I2S.setAllPins(sck, fs,
```

To inialitize the library, use the `begin()` method:

```
1  I2S.begin(mode, sampleR
```

To read data, use the `read()` method, which will return the last sample.

```
1  I2S.read()
```

available in the Board Package under **Examples > I2S**.

Further reading:

- I2S API docs (Espressif)
- I2S Reference (Espressif)

Dual Core

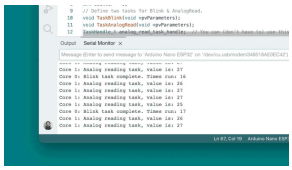
The ESP32-S3 is based on the dual-core XTensa LX7, which can run code separately on two cores. This is enabled through FreeRTOS, by setting up tasks that run on each core (similarly to how `void loop()` is implemented). The cores available are `0` and `1`.

The example below is a modified version of the BasicMultiThreading example found in the Arduino ESP32 Board Package, and demonstrates how to use two common operations simultaneously:

- Blink an LED using one task on a specific core (0),
- Read an analog pin using a second task on a specific core (1).

```
/* Basic Multi Threa

   Modified 16th Oct

   Set up two tasks
   one that blinks a

   These tasks will

   This example code
   Unless required b
   software is distr
   CONDITIONS OF ANY
*/

// Define the cores
#define CORE_0 0
#define CORE_1 1

#define ANALOG_INPUT
#define LED_BUILTIN


int counter = 0;
// Define two tasks
void TaskBlink(void
void TaskAnalogRead(
TaskHandle_t analog_
```

When running this example, open the Serial Monitor tool and you will see what happens on each core.



Dual core example.

The task is created in the `xTaskCreatePinnedToCore()`, inside `xTaskCreatePinnedToCore()` we specify a number of

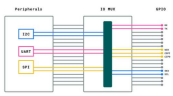
code inside task functions are placed inside the `for (;;){}` statement, that will loop infinitely.

More information about dual-core on the ESP32 along with a detailed explanation of the example is available at **Basic Multi Threading Example**.

IO Mux & GPIO Matrix

The ESP32-S3 SoC features an IO mux (input/output multiplexer) and a GPIO matrix. The IO mux acts as a data selector and allows for different peripherals to be connected to a physical pin.

The ESP32-S3 chip has 45 physical GPIOs, but many more digital peripherals. The IO mux provides the flexibility of routing the signals to different GPIOs, thus changing the function of a specific pin.



Peripheral IO MUX

This technique is well known and applied within ESP32 boards, but on the Nano ESP32 we use a set of default pins for the I2C, SPI & UART peripherals to remain consistent with previous designs.

As an example, the Nano ESP32's

be changed to e.g. D8,D9 if you need to use another set of pins. This is done through the mux / GPIO matrix.

Re-Assigning Pins

You can read more about re-assigning the peripherals through the links below:

- I2C configuration (link to Espressif docs)
- UART configuration (link to Espressif docs)

You can also read Espressifs technical reference manual here:

- IO MUX and GPIO Matrix (ESP32-S3 technical reference manual)

Wi-Fi®

The Nano ESP32 has a NORA-W106 module which has the ESP32-S3 SoC embedded. This module supports Wi-Fi® communication over the 2.4 GHz band.

There are several examples provided bundled with the Board Package that showcase how to make HTTP requests, host web servers, send data over MQTT etc.

RGB

The ESP32 features an RGB LED that can be controlled with the `LED_RED` `LED_GREEN` and

are not accessible on the headers of the board, and can only be used for the RGB LED.

> Some boards from the first limited production batch were assembled with a different RGB LED which has the green and blue pins inverted. Read our full Help Center article here

To control them, use:

```
digitalWrite(LED_RED, S
digitalWrite(LED_GREEN,
digitalWrite(LED_BLUE,
```

These pins are so called active-low, what this means in practice is that to turn on one of the LEDs, you need to write it to `LOW`, like this:

```
digitalWrite(LED_RED, L
```

USB HID

Nano ESP32 can be used to emulate an HID device by using e.g. `Mouse.move(x,y)` or `Keyboard.press('w')`.

These are minimal examples for keyboard and mouse use:

Keyboard Example

```
#include "USB.h"
#include "USBHIDKeyboa
USBHIDKeyboard Keyboar

void setup() {
  // put your setup co
  Keyboard.begin();
  USB.begin();
}

void loop() {
  // put your main cod
  Keyboard.print("Hell
  delay(500);
}
```

Mouse Example

```
#include "USB.h"
#include "USBHIDMouse.
USBHIDMouse Mouse;

void setup() {
  // put your setup co
  Mouse.begin();
  USB.begin();
}

void loop() {
  // put your main cod
  Mouse.move(5, 0, 0);
  delay(50);
}
```

Several ready to use examples are also available in the Board Package at **Examples > USB**.

Remember that if the board stops being recognised in the IDE, you can put it in **Arduino Bootloader Mode** to recover it.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?